Modeling of Short Amino Acid Sequences Using Direct Coupling Analysis and Gibbs Sampling

Spring 2026

Emily Guan

Mentor: Dr. Joel S. Bader

Abstract

This paper implements and evaluates a Direct Coupling Analysis model for generative amino acid sequence modeling. A pairwise Potts model is trained on a multiple sequence alignment to learn site-specific amino acid preferences and pairwise residue dependencies. New sequences are generated using Gibbs sampling, and the resulting samples are evaluated against the training distribution. Results show that the model can generate diverse sequences while preserving many first-order statistics of the training MSA, though further *in vitro* evaluation is needed to determine whether any ofa the generations are biologically feasible.

Contents

1 Introduction **4**
1.1 Motivation . . . 4
1.2 Problem Statement . . . 4
1.3 Research Questions . . . 4

2 Background **5**
2.1 Previous and Related Work . . . 5
2.2 Implementation Inspiration from `Hopf et al.` . . . 5

3 Data **6**
3.1 Dataset . . . 6
3.2 Preprocessing . . . 7
3.3 Sequence Reweighting . . . 7

4 Training Methodology **8**
4.1 Model Definition . . . 8
4.2 Training Objective . . . 8
4.3 Regularization . . . 9
4.4 Symmetry Problem . . . 9

5 Sampling Methodology **10**
5.1 Why Sample . . . 10
5.2 Gibbs Sampling . . . 10
5.3 Batched Gibbs . . . 10
5.4 Burn In . . . 10
5.5 Initialization . . . 10

6 Evaluation Methodology **11**
6.1 Training Evaluation . . . 11
6.1.1 NLL . . . 11
6.1.2 Perplexity . . . 11
6.1.3 Conditional Accuracy . . . 11
6.2 Generated Samples Evaluation . . . 11
6.2.1 Diversity . . . 11
6.2.2 Pairwise Hamming . . . 12
6.2.3 Position-Wise Entropy . . . 12
6.2.4 Energy Distribution . . . 12
6.2.5 First-Order Statistics: Amino Acid Frequency by Position . . . 12
6.2.6 Pairwise Statistics . . . 13
6.2.7 Mutual Information . . . 13

7 Results **14**
7.1 Training Specs . . . 14
7.2 Validation Performance Table . . . 14
7.3 Sampling Diversity . . . 14
7.4 Entropy Preservation . . . 15
7.5 Frequency Preservation . . . 16
7.6 Train-Match Similarity . . . 17
7.7 Potts Scoring . . . 17

8 Discussion **19**
8.1 Limitations . . . 19
8.1.1 *in vitro* Testing . . . 19
8.1.2 Sequence Length . . . 19
8.1.3 Small Datasets . . . 19
8.2 Future Work . . . 19
8.3 Conclusion . . . 20

1 Introduction

1.1 Motivation

Proteins perform essential biological functions, yet their amino-acid sequences are highly variable. Since evolution has already sampled many variants of a protein family, a multiple sequence alignment (MSA) may contain statistical signals for folding, stability, and function. Inverse statistical physics uses this idea by treating natural protein sequences as samples from an underlying probability distribution and attempting to infer the constraints that shape that distribution. Potts modeling allows us to infer both site-specific amino-acid preferences and pairwise dependencies between positions.

Lower level (atomic/molecular) simulations can be accurate, but they are computationally expensive and difficult to apply across large protein families. In contrast, sequence-based models exploit the rapid growth of sequencing databases. Prior work has shown that DCA-style models can predict residue contacts. For example, bmDCA was used to design artificial chorismate mutase enzymes with catalytic function in vivo[1].

1.2 Problem Statement

In this paper, I will be repurposing a pairwise Potts model based on Hopf et al.[2] trained on an MSA to serve as a generative model for short amino-acid sequences. A protein sequence of length L can be represented as

$$a = (\sigma_1, \sigma_2, \ldots, \sigma_L), \tag{1}$$

where each σ_i is an amino acid at position i. The probability of a sequence is modeled as

$$P(\sigma) = \frac{1}{Z} \exp\left(-H(\sigma)\right), \tag{2}$$

where Z is the partition function. The Hamiltonian is defined as

$$H(\sigma) = -\sum_i h_i(\sigma_i) - \sum_{i<j} J_{ij}(\sigma_i, \sigma_j), \tag{3}$$

where $h_i(\sigma_i)$ represents the site-specific preference for amino acid σ_i at position i, and $J_{ij}(\sigma_i, \sigma_j)$ represents the coupling between amino acids at positions i and j. These terms allow the model to probe how an amino acid at one position may depend on amino acids at other positions.

However, even if a model assigns low energy to sequences, it is not automatically a good generator. A useful generative model should produce new sequences that resemble the training distribution without simply memorizing it. This project therefore focuses on evaluating whether sequences generated by Gibbs sampling preserve important statistical properties of the original MSA.

1.3 Research Questions

This project is guided by the following research questions:

1. Can a pairwise Potts model trained on an MSA generate amino-acid sequences that preserve single-site amino-acid statistics and pairwise dependencies?

2. How do the generated sequences compare to natural/training sequences under the model's energy function?

2 Background

2.1 Previous and Related Work

This project builds on several lines of prior work connecting Direct Coupling Analysis and protein design.

Cocco et al.[3] provides most of the mathematical and theoretical foundation used in this project by framing protein sequence analysis as an inverse statistical physics problem. In this view, an MSA is treated as a sample from an unknown Boltzmann-like distribution over protein sequences, and the goal is to infer a simplified Hamiltonian that captures the constraints of the protein family. Many of the formulations used in this project can be found in Cocco et al[3].

Earlier work from Ranganathan and collaborators[1] showed that pairwise covariation can be sufficient to generate artificial protein sequences with natural-like properties. In their experiments, artificial sequences were generated by preserving only independent conservation, or both conservation and pairwise covariation. Sequences preserving only independent conservation did not fold correctly, whereas a fraction of sequences preserving pairwise covariation did fold. This result suggests that protein family constraints are not contained only in individual residues, but also in residue-residue dependencies.

Hopf et al.[2] extended this idea to mutation-effect prediction through EVmutation. Their model treats each protein family as a probability distribution and uses an energy function with both site-specific fields and pairwise couplings. Mutation effects were predicted by the change in statistical energy between the mutant and wild-type sequence.

Together, these works motivate the central question of this project: whether a Potts model trained on an MSA can be used as a meaningful sequence generator. This project evaluates whether Gibbs sampling from the learned energy function produces sequences that preserve the statistical structure of the original MSA.

2.2 Implementation Inspiration from `Hopf et al.`

This project draws several implementation and modeling ideas from Hopf et al.[2] and the related `plmc`[4] implementation from the Debora Marks lab.

First, Hopf et al. motivate the use of a pairwise Potts model for biological sequences. This project follows the same basic modeling choice: a generated sequence should not only match the amino-acid frequencies at each position, but should also preserve pairwise relationships learned from the MSA.

Second, Hopf et al. use maximum pseudolikelihood instead of exact maximum likelihood. Exact likelihood would require normalizing over the full protein sequence space, which is computationally infeasible. Pseudolikelihood avoids this by training the model to predict each amino acid from the rest of the sequence.

Third, Hopf et al. use regularization to reduce overfitting. This is especially important for Potts models because the number of pairwise coupling parameters can be large compared with the number of available sequences. This project follows the same principle by applying separate regularization strengths to the field parameters and coupling parameters.

The main difference is the final use of the learned model. Hopf et al. focus on predicting mutation effects by comparing the statistical favorability of mutant and wild-type sequences. In this project, the learned Potts model is instead used as a generator. After training, Gibbs sampling is used to produce new sequences, and those sequences are evaluated for diversity, amino-acid frequency preservation, pairwise statistics, mutual information, and Potts score distributions.

3 Data

3.1 Dataset

The dataset used for this project comes from Dr. Joel S. Bader.

The dataset is stored in tabular format, with each row representing one unique amino-acid sequence and its associated count and weighting information. A sample of the data format is shown below:

AA_SEQ	AA_CNT	AA_WEIGHT	NT_UNIQ
AESFVEWDSW	1	1.0	1
APQYVEWDEW	1	1.0	1
DCTYVEWDEW	3	2.0	1
DSTYVEWDEW	1	1.0	1

(4)

The data are split across five files.

File	Number of Lines
NGS-161.00	28,506
NGS-161.01	297
NGS-165.00	62,252
NGS-165.01	883
NGS-166.00	314,490

Table 1: Summary of dataset file sizes.

Column	Description
AA_SEQ	Amino-acid sequence of length $L = 10$. In this project, sequences have no gaps, so gap characters are not modeled as an additional state.
AA_CNT	Count of the amino-acid sequence in the dataset.
AA_WEIGHT	Precomputed sequence weight used to account for redundancy.
NT_UNIQ	Number of unique nucleotide-level variants associated with the amino-acid sequence, since multiple nucleotide sequences can encode the same amino-acid sequence.

Table 2: Description of columns in the input files.

3.2 Preprocessing

The preprocessing step converts raw amino-acid strings into a numerical representation that can be used by the Potts model. Each sequence is assumed to contain only the 20 standard amino acids,

$$\mathcal{A} = \{\texttt{A},\texttt{C},\texttt{D},\texttt{E},\texttt{F},\texttt{G},\texttt{H},\texttt{I},\texttt{K},\texttt{L},\texttt{M},\texttt{N},\texttt{P},\texttt{Q},\texttt{R},\texttt{S},\texttt{T},\texttt{V},\texttt{W},\texttt{Y}\}, \tag{5}$$

so the alphabet size is 20.

Since the dataset contains no gaps, gaps or empty alignment states are not modeled as an additional amino-acid category. Therefore, the alphabet is restricted to the 20 standard amino acids rather than the 21-state alphabet sometimes used in DCA models.

Each amino acid $a \in \mathcal{A}$ is mapped to an integer index:

$$\texttt{A} \mapsto 0, \quad \texttt{C} \mapsto 1, \quad \ldots, \quad \texttt{Y} \mapsto 19. \tag{6}$$

After preprocessing, the μ-th sequence is represented as

$$x^{\mu} = (x_1^{\mu}, x_2^{\mu}, \ldots, x_L^{\mu}), \tag{7}$$

3.3 Sequence Reweighting

Sequence reweighting is used because an MSA is not an independent and uniformly sampled collection of protein sequences. Closely related homologs may appear many times because of evolutionary relatedness or database sampling bias. If all sequences were treated equally, highly redundant groups could dominate the learned fields and couplings. Hopf et al.[2] describe this issue and use sequence reweighting to reduce the effect of highly similar sequences.

In the provided data, each sequence x^{μ} already has an associated weight column, `AA_WEIGHT`. The original weight for sequence μ is denoted by w_{μ}.

After capping, the weights are normalized:

$$w_{\mu}^{\text{norm}} = \frac{w_{\mu}^{\text{cap}}}{\frac{1}{N}\sum_{\nu=1}^{N} w_{\nu}^{\text{cap}}}. \tag{8}$$

This normalization makes the mean sequence weight equal to one. The final weighted training objective therefore uses the encoded sequence dataset X from Equation **??** and the normalized weights w_{μ}^{norm} from Equation 8. These weights are used in the pseudolikelihood loss so that each sequence contributes proportionally to its adjusted statistical importance.

4 Training Methodology

4.1 Model Definition

The model used in this project is a symmetric pairwise Potts model trained by weighted site-factored pseudolikelihood. Each protein sequence is represented as an encoded vector (7) where x^μ denotes the μ-th sequence, $L = 10$ is the sequence length, and each $x_i^\mu \in \mathcal{A}$ is the amino acid at position i.

The model contains two sets of trainable parameters. The first is a field matrix

$$h \in \mathbb{R}^{L\times q}, \tag{9}$$

where $h_i(a)$ represents the site-specific preference for amino acid $a \in \mathcal{A}$ at position i. The second is a coupling tensor

$$J \in \mathbb{R}^{L\times L\times q\times q}, \tag{10}$$

where $J_{ij}(a, b)$ represents the pairwise compatibility between amino acid a at position i and amino acid b at position j. In the implementation, the fields are initialized to zero, while the raw coupling tensor is initialized with small random values scaled by 0.01.

For a sequence x, the model energy is

$$E(x) = \sum_{i=1}^{L} h_i(x_i) + \sum_{1\le i<j\le L} J_{ij}(x_i, x_j). \tag{11}$$

This energy can be used to score sequences as seen later in Section 6.2.4. For example, mutation effects can be computed by comparing the energy of a mutant sequence to the energy of a wild-type sequence, as in Hopf et al.[2]:

$$\Delta E = E(x_{\text{mut}}) - E(x_{\text{wt}}). \tag{12}$$

4.2 Training Objective

Exact maximum-likelihood training of a Potts model is difficult because it requires computing the partition function over all possible sequences, which is computationally infeasible. Instead, the model is trained using pseudolikelihood.

For each position i, the model predicts the amino acid at that position conditioned on all other positions in the sequence:

$$\ell_i(a \mid x_{j\neq i}^\mu) = h_i(a) + \sum_{j\neq i} J_{ij}(a, x_j^\mu), \tag{13}$$

These logits are converted into conditional probabilities using the softmax function:

$$P_i(x_i^\mu = a \mid x_{j\neq i}^\mu) = \frac{\exp\left(\ell_i(a \mid x_{j\neq i}^\mu)\right)}{\sum_{c\in\mathcal{A}} \exp\left(\ell_i(c \mid x_{j\neq i}^\mu)\right)}. \tag{14}$$

The weighted negative log-pseudolikelihood is then

$$\mathcal{L}_{\text{NPL}} = -\sum_{\mu=1}^{N} w_\mu^{\text{norm}} \sum_{i=1}^{L} \log P_i(x_i^\mu \mid x_{j\neq i}^\mu), \tag{15}$$

where w_μ^{norm} is the normalized training weight for sequence μ from Equation 8.

Training is performed with the L-BFGS optimizer, inspired by xqDing's implementation of DCA[5]. The optimizer minimizes the total regularized loss.

4.3 Regularization

Because the Potts model contains many parameters, regularization is necessary. The number of field parameters is Lq and the number of pairwise coupling parameters, ignoring symmetry, scales as L^2q^2. Without regularization, the model could overfit rare amino-acid combinations or assign excessively large coupling values.

The implementation uses L2 regularization on both fields and couplings, as in Hopf et al.[2], making the total training objective:

$$\mathcal{L} = \mathcal{L}_{\text{NPL}} + \lambda_h \sum_{i=1}^{L} \sum_{a \in \mathcal{A}} h_i(a)^2 + \lambda_J \sum_{1 \le i < j \le L} \sum_{a \in \mathcal{A}} \sum_{b \in \mathcal{A}} J_{ij}(a,b)^2. \tag{16}$$

where $\mathcal{L}_{\text{NPL}}$ is Equation 15. Here, λ_h controls regularization strength for the field parameters, and λ_J controls regularization strength for the coupling parameters.

The default configuration uses $\lambda_h = 10^{-3}, \lambda_{J,\text{scale}} = 10^{-2}$. The code supports two choices for the coupling regularization value. The first uses the unscaled value

$$\lambda_J = \lambda_{J,\text{scale}}. \tag{17}$$

The second uses a Hopf-inspired scaling:

$$\lambda_J = \lambda_{J,\text{scale}} \cdot q \cdot (L-1). \tag{18}$$

The scaled version increases the coupling penalty with alphabet size and sequence length, which is useful because the number of coupling parameters grows rapidly as L increases.

After training, the implementation reports the negative log-pseudolikelihood, the field regularization term, the coupling regularization term, and the norms of h and J.

4.4 Symmetry Problem

A pairwise Potts model should have symmetric couplings. (interaction between position i and position j should be the same interaction as the one between position j and position i). Therefore, the coupling tensor should satisfy:

$$J_{ij}(a,b) = J_{ji}(b,a). \tag{19}$$

The implementation stores a raw coupling tensor, J_{raw} and constructs a symmetrized coupling tensor before using it in the loss:

$$J_{ij}(a,b) = \frac{1}{2}\left[J_{\text{raw},ij}(a,b) + J_{\text{raw},ji}(b,a)\right]. \tag{20}$$

This transpose swaps both the site indices (i,j) and the amino-acid indices (a,b). The diagonal blocks are then set to zero:

$$J_{ii}(a,b) = 0 \qquad \text{for all } a,b \in \mathcal{A}. \tag{21}$$

This prevents a position from coupling to itself and ensures that all pairwise interactions are represented consistently.

5 Sampling Methodology

5.1 Why Sample

After training, the Potts model defines an energy function over full amino-acid sequences. Sampling is used to test whether the trained model behaves as a generative model. Fitting the model only shows that it can assign conditional probabilities to training sequences. Sampling asks a stronger question: if we draw new sequences from the learned distribution, do those sequences resemble the training MSA?

5.2 Gibbs Sampling

The sampling method used in this project is Gibbs sampling. Gibbs sampling updates one position at a time while holding the rest of the sequence fixed. For a sequence the sampler chooses a site i and samples a new amino acid from the conditional distribution (Equation 14).

At each Gibbs update, the sampler:

1. reads the logits for site i,
2. divides the logits by the temperature T,
3. applies a softmax to obtain amino-acid probabilities,
4. samples a new residue,
5. updates the sequence and adjusts the cache.

If the sampled residue is the same as the old residue, the cache does not need to be changed. If the residue changes, the cache is updated by adding the contribution of the new residue and subtracting the contribution of the old residue:

$$\texttt{cache} \leftarrow \texttt{cache} + J_{\cdot i}(\cdot, x_i^{\text{new}}) - J_{\cdot i}(\cdot, x_i^{\text{old}}).$$

5.3 Batched Gibbs

The implementation uses a batched version of Gibbs sampling to make sampling faster and more stable. Instead of sampling one sequence at a time, it maintains a batch of B parallel chains. This allows the project to generate thousands of sequences efficiently while still using single-site Gibbs updates. Each chain is an independent sequence being sampled from the same Potts model. This single-site Gibbs updates across a batch of independent chains, and gives the computational benefit of parallelism while preserving the simple conditional update rule for one site at a time.

5.4 Burn In

Burn-in is used to discard early samples before we move toward the model distribution. The initial sequences are random, so the first few Gibbs sweeps may reflect the initialization more than the learned distribution. To reduce this effect, the sampler performs extra updates before storing samples. For example, when `burn_in_frac = 0.1`, approximately the first 10% of the full sampling trajectory is discarded.

5.5 Initialization

The sampler initializes each chain randomly:

$$x_{bi} \sim \text{Uniform}\{0, \ldots, q-1\},$$

creating a batch of random amino-acid sequences. After initialization, the sampler builds the cache once using the current sequences. The cache is then updated incrementally during Gibbs sampling (see batched Gibbs Sampling 5.3).

6 Evaluation Methodology

6.1 Training Evaluation

Evaluation is used to determine whether the trained Potts model is both a good conditional model of the training data and a useful generator of new sequences.

6.1.1 NLL

For a sequence x^μ, the model predicts each amino acid x_i^μ from the remaining positions $x_{j\neq i}^\mu$. The average negative log-pseudolikelihood per site is

$$\text{NLL/site} = -\frac{1}{LW}\sum_{\mu=1}^{N} w_\mu \sum_{i=1}^{L} \log P_i(x_i^\mu \mid x_{j\neq i}^\mu), \tag{22}$$

where

$$W = \sum_{\mu=1}^{N} w_\mu. \tag{23}$$

Lower NLL/site means the model assigns higher conditional probability to the observed amino acids.

6.1.2 Perplexity

Perplexity (drawn from NLP) is computed as

$$\text{Perplexity} = \exp(\text{NLL/site}). \tag{24}$$

Lower perplexity means the model is less uncertain when predicting positions.

6.1.3 Conditional Accuracy

Conditional accuracy is the weighted fraction of positions where the model's most likely amino acid matches the observed amino acid:

$$\text{Accuracy} = \frac{1}{LW}\sum_{\mu=1}^{N} w_\mu \sum_{i=1}^{L} \mathbf{1}\left[\hat{x}_i^\mu = x_i^\mu\right], \tag{25}$$

where

$$\hat{x}_i^\mu = \arg\max_{a\in\mathcal{A}} P_i(a \mid x_{j\neq i}^\mu). \tag{26}$$

Higher conditional accuracy means the model more often predicts the observed amino acid at each position.

6.2 Generated Samples Evaluation

6.2.1 Diversity

Diversity measures whether the generated sequences are varied or whether the sampler collapses to a small number of repeated sequences. Let X_{gen} contain G generated sequences. The unique fraction is

$$\text{Unique Fraction} = \frac{|X_{\text{gen}}^{\text{unique}}|}{G}. \tag{27}$$

A value close to 1 means that most generated sequences are distinct, while a value close to 0 means that the sampler repeatedly generated the same few sequences. The evaluation script also produces a duplicate report. This report lists the most common repeated generated sequences and their counts.

6.2.2 Pairwise Hamming

Mean pairwise Hamming distance measures the average normalized difference between pairs of generated sequences. For two sequences x and y, the normalized Hamming distance is

$$d_H(x, y) = \frac{1}{L} \sum_{i=1}^{L} \mathbf{1}[x_i \neq y_i]. \tag{28}$$

The evaluation also computes nearest-training Hamming distances. For each generated sequence $x^{(g)}$, it finds the closest sequence in the training set:

$$d_{\text{nearest}}(x^{(g)}) = \min_{x \in X_{\text{train}}} d_H(x^{(g)}, x). \tag{29}$$

The train-match fraction is the fraction of generated sequences that exactly appear in the training set:

$$\text{Train Match Fraction} = \frac{1}{G} \sum_{g=1}^{G} \mathbf{1}\left[d_{\text{nearest}}(x^{(g)}) = 0\right]. \tag{30}$$

6.2.3 Position-Wise Entropy

Position-wise entropy measures how diverse the amino-acid distribution is at each sequence position. For position i, the entropy is

$$H_i = -\sum_{a \in \mathcal{A}} f_i(a) \log f_i(a). \tag{31}$$

The entropy ratio compares generated and reference entropy:

$$\text{Entropy Ratio} = \frac{\overline{H}_{\text{gen}}}{\overline{H}_{\text{ref}}}. \tag{32}$$

A ratio close to 1 means that generated sequences have a similar level of position-wise diversity as the reference data.

6.2.4 Energy Distribution

Energy distribution evaluates whether generated sequences receive model scores similar to real training or validation sequences. The implementation computes a Potts score for each sequence using Equation 11. Higher scores indicate higher model preference. Scores are compared across training, validation, generated, and random sequences. Ideally, generated sequences should score closer to training and validation sequences than to random sequences.

6.2.5 First-Order Statistics: Amino Acid Frequency by Position

Amino-acid frequency by position measures the first-order distribution of residues at each aligned site. For a dataset X, the frequency of amino acid a at position i is written as

$$f_i^X(a) = \frac{\sum_{x \in X} w(x) \mathbf{1}[x_i = a]}{\sum_{x \in X} w(x)}. \tag{33}$$

The evaluation compares generated and reference single-site frequencies using mean absolute error:

$$\text{Single Freq MAE} = \frac{1}{Lq} \sum_{i=1}^{L} \sum_{a \in \mathcal{A}} \left| f_i^{\text{ref}}(a) - f_i^{\text{gen}}(a) \right|. \tag{34}$$

A lower value means generated residue frequencies are closer to the reference dataset.

6.2.6 Pairwise Statistics

Pairwise statistics measure how often pairs of amino acids occur together at pairs of positions. For a dataset X, the pair frequency is

$$f_{ij}^{X}(a,b) = \frac{\sum_{x \in X} w(x)\mathbf{1}[x_i = a]\mathbf{1}[x_j = b]}{\sum_{x \in X} w(x)}. \tag{35}$$

The evaluation compares generated and reference pairwise frequencies using pairwise mean absolute error:

$$\text{Pair Freq MAE} = \frac{1}{\binom{L}{2} q^2} \sum_{i<j} \sum_{a \in \mathcal{A}} \sum_{b \in \mathcal{A}} \left| f_{ij}^{\text{ref}}(a,b) - f_{ij}^{\text{gen}}(a,b) \right|. \tag{36}$$

These metrics test whether generated sequences preserve pairwise co-occurrence patterns from the reference data.

6.2.7 Mutual Information

Mutual information measures statistical dependence between two positions. For positions i and j, mutual information is

$$MI_{ij} = \sum_{a,b} f_{ij}(a,b) \log \frac{f_{ij}(a,b)}{f_i(a) f_j(b)}. \tag{37}$$

If two positions are independent, then

$$f_{ij}(a,b) = f_i(a) f_j(b), \tag{38}$$

and the mutual information is approximately zero. Larger values indicate stronger statistical dependence.

7 Results

7.1 Training Specs

Dataset	Sample Size	Train Size	Val Size	CPU Runtime
NGS-161.00	28,506	25,655	2,851	1m 39s
NGS-161.01	297	267	30	3.66s
NGS-165.00	62,252	56,027	6,225	3m 38s
NGS-165.01	883	795	88	5.54s
NGS-166.00	314,490	283,041	31,449	19m 02s

Table 3: Dataset sizes, train/validation splits, and training runtime for each Potts model.

7.2 Validation Performance Table

Dataset	Val NLL/site	Val Perplexity	Val Accuracy
NGS-161.00	1.077	2.935	0.659
NGS-161.01	0.672	1.958	0.839
NGS-165.00	1.131	3.098	0.663
NGS-165.01	0.744	2.105	0.837
NGS-166.00	1.346	3.841	0.588

Table 4: Validation performance across trained Potts models. Lower NLL/site and perplexity indicate better conditional fit, while higher validation accuracy indicates better amino-acid prediction.

Table 4 summarizes the validation NLL/site from Equation 22, perplexity from Equation 24, and conditional accuracy from Equation 25 for each trained Potts model.

The smaller `.01` datasets achieve lower NLL/site, lower perplexity, and higher validation accuracy than their corresponding `.00` datasets, suggesting that these subsets are easier to model or contain less variability. Among the full-scale datasets, `NGS-166.00` has the weakest fit, with the highest perplexity and lowest validation accuracy, which is reflective of the difficulty of modeling larger datasets.

7.3 Sampling Diversity

Figure 1 shows the generated-sequence unique fraction from Equation 27 for each dataset. The full-size `.00` datasets produced highly diverse samples, suggesting that the larger-input models sample from a much broader sequence distribution than the smaller-input models.

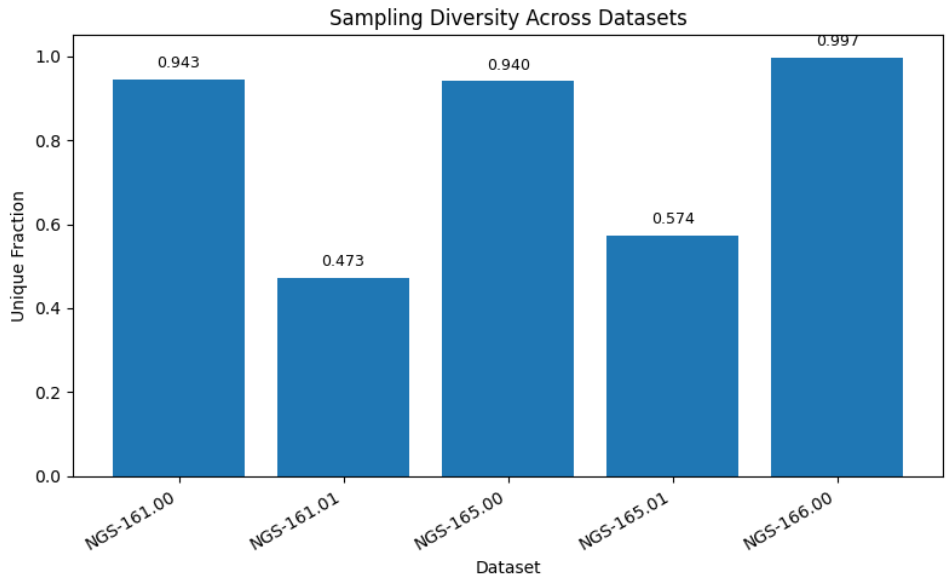


Figure 1: Unique fraction of generated sequences across datasets. Higher values indicate greater sampling diversity, while lower values indicate that the Gibbs sampler generated more repeated sequences.

7.4 Entropy Preservation

Figure 2 shows the entropy ratio from Equation 32, comparing generated mean position entropy to training mean position entropy.

The generated sequences preserved position-wise entropy well across all datasets. Entropy ratios ranged from 0.965 to 0.990, meaning that the generated sequences had slightly lower entropy than the training data but remained close overall. This indicates that Gibbs sampling did not collapse completely to a few sequences; instead, the generated samples generally maintained the position-wise diversity of the original data.

Across all datasets, generated entropy remains close to training entropy, which indicates that Gibbs sampling preserves much of the position-wise diversity learned by the model.

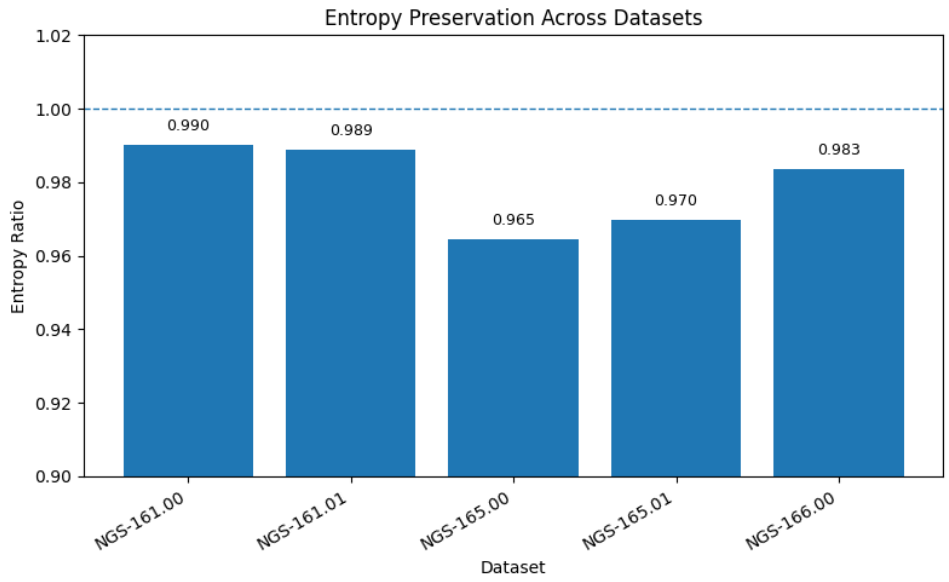


Figure 2: Entropy ratio between generated and training sequences across datasets. Values close to 1 indicate that Gibbs sampling preserved the position-wise diversity of the training distribution.

7.5 Frequency Preservation

Dataset	Single Freq. MAE	Pair Freq. MAE	MI Corr.
NGS-161.00	0.0018	0.00021	0.988
NGS-161.01	0.0028	0.00035	0.991
NGS-165.00	0.0026	0.00026	0.967
NGS-165.01	0.0042	0.00042	0.993
NGS-166.00	0.0031	0.00036	0.987

Table 5: Frequency preservation metrics comparing generated sequences to the training data. Single-frequency MAE measures preservation of amino-acid frequencies at individual positions, pair-frequency MAE measures preservation of pairwise amino-acid co-occurrence frequencies, and MI correlation measures preservation of mutual information structure.

Table 5 summarizes the single-frequency MAE from Equation 34, pair-frequency MAE from Equation 36, and mutual information correlation based on Equation 37. These metrics evaluate whether generated sequences preserve the first-order and pairwise statistical structure of the training data.

The first-order and pairwise frequency metrics indicate that the generated sequences closely match the training distribution. Single-site frequency MAE values are all below 0.005, and pairwise frequency MAE values are all below 0.0005. This suggests that the generated samples preserve both individual amino-acid preferences and pairwise amino-acid co-occurrence patterns. The mutual information correlations are also high for all datasets, ranging from 0.967 to 0.993, showing that the generated sequences preserve much of the dependency structure between positions.

7.6 Train-Match Similarity

Dataset	Train-Match Fraction	Nearest Hamming	Pairwise Hamming
NGS-161.00	0.253	0.097	0.570
NGS-161.01	0.370	0.082	0.399
NGS-165.00	0.333	0.087	0.582
NGS-165.01	0.404	0.080	0.469
NGS-166.00	0.113	0.158	0.807

Table 6: Novelty and diversity metrics for generated sequences. Train-match fraction measures the fraction of generated sequences that exactly appear in the training set, nearest-training Hamming measures the average distance from each generated sequence to its closest training sequence, and pairwise Hamming measures diversity among generated sequences.

Table 6 summarizes novelty and diversity metrics for generated sequences. Train-match fraction is defined in Equation 30, nearest-training Hamming distance is defined in Equation 29, and pairwise Hamming diversity is based on the normalized Hamming distance in Equation 28.

The novelty and diversity results show clear differences between the full datasets and the smaller `.01` datasets. The full datasets, especially `NGS-166.00`, generated more diverse sequences. `NGS-166.00` had a unique fraction of 0.997, a mean pairwise Hamming distance of 0.807, and the lowest exact train-match fraction at 0.113. This indicates that the model generated mostly novel and diverse sequences rather than simply reproducing training examples.

By contrast, the smaller `.01` datasets had lower unique fractions and higher exact train-match fractions. This suggests that the sampler for the smaller datasets is more concentrated around a smaller set of common or high-probability sequences, indicating less novelty.

7.7 Potts Scoring

Figure 3 compares the learned Potts score from Equation **??** across training, validation, generated, and random sequences for each dataset.

The score distributions provide strong evidence that the generated sequences are model-like rather than random. In every dataset, generated sequences have mean scores close to train and validation sequences, while random sequences have mean scores near zero. For example, in `NGS-161.00`, the generated score mean is 42.40, close to the train mean of 41.34, while the random mean is -0.06. Similarly, in `NGS-165.01`, the generated score mean is 59.53, close to the train mean of 59.58, while the random mean is -0.08. This separation suggests that the Potts model learned meaningful sequence constraints and that Gibbs sampling produces sequences that satisfy those constraints.

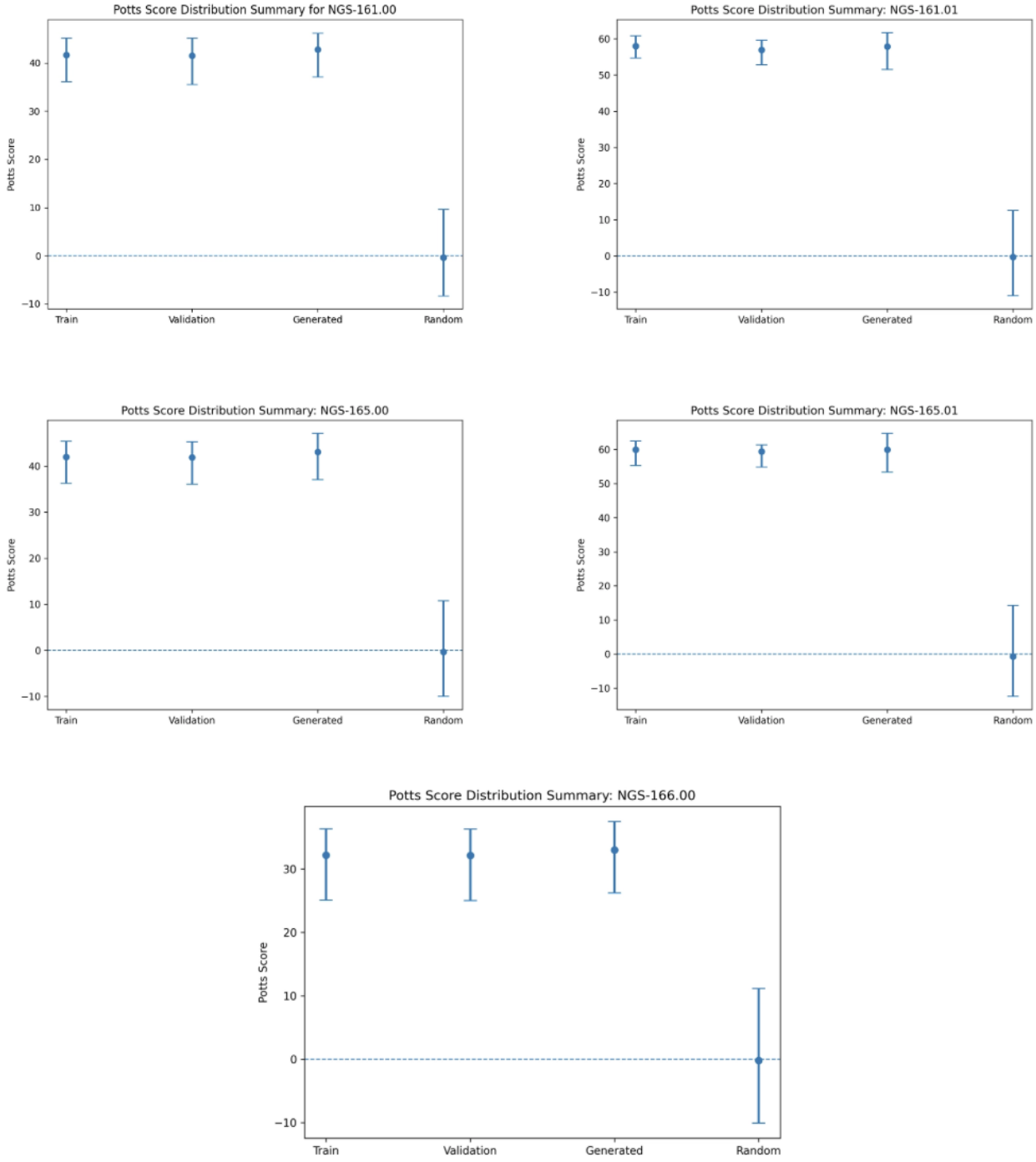


Figure 3: Potts score distribution summaries across five NGS datasets. Points show median scores, and error bars show the 5th–95th percentile range.

8 Discussion

Overall, the evaluation supports the main claim of the project: the trained Potts models can generate sequences that resemble the training distribution in both first-order and pairwise statistics. The generated samples preserve amino-acid frequencies, maintain high mutual information correlation, and receive model scores similar to real sequences. The main limitation is that novelty varies by dataset. The full datasets produce highly diverse and mostly novel samples, while the smaller `.01` datasets produce more repeated sequences and a higher fraction of exact training matches. Therefore, the models are valid generators, but their diversity depends strongly on the size and entropy of the dataset.

The frequency preservation results show that the generated sequences match the training distribution well. Single-frequency MAE values are low across all datasets, meaning that the generated samples reproduce basic position-wise amino-acid preferences. Pairwise frequency MAE values are also low, suggesting that the model preserves amino-acid co-occurrence patterns between positions.

Mutual information preservation is one of the strongest results. Since the main advantage of a Potts/DCA model over an independent-site model is its ability to model pairwise dependencies through $J_{ij}(a, b)$, the high MI correlations suggest that the learned couplings capture meaningful dependency structure.

The Potts score results also support the quality of the generated samples. Across all datasets, generated sequences score similarly to train and validation sequences and far above random sequences. This indicates that Gibbs sampling produces sequences that satisfy the learned Potts constraints rather than random amino-acid strings.

8.1 Limitations

8.1.1 in vitro Testing

One of the most obvious limitations is that the generated sequences preserve MSA-like properties, but this does not prove that they fold correctly or perform a biological function. Experimental validation or external biological scoring would be needed to make claims about function.

8.1.2 Sequence Length

A second limitation is sequence length. All sequences have length $L = 10$, which makes the project computationally manageable but much shorter than most real protein domains. Longer sequences would require many more coupling parameters, stronger regularization, and more careful sampling.

8.1.3 Small Datasets

The smaller `.01` datasets show weaker generative behavior. They have lower unique fractions and higher train-match fractions, suggesting that the sampler more often revisits training-like sequences. This indicates that dataset size and diversity strongly affect generation quality.

8.2 Future Work

This approach should be tested on longer protein sequences. This would make the task more biologically realistic, but would require larger MSAs, gap handling (21st "alphabet" letter), stronger regularization, and more efficient computation.

Finally, generated sequences should eventually be evaluated using external biological criteria, such as structure prediction, protein language model scoring, mutation-effect prediction, motif analysis, or experimental assays. These evaluations would test whether statistically realistic sequences are also biologically plausible or functional.

8.3 Conclusion

This project implemented a Potts/DCA-based sequence generation pipeline using weighted pseudolikelihood training and Gibbs sampling. The results show that the model can generate sequences that preserve important statistics of the original data, including amino-acid frequencies, pairwise co-occurrence patterns, entropy, mutual information, and Potts score structure.

Larger datasets produce stronger results, with higher diversity and lower train-match behavior. The smaller `.01` datasets were useful for testing but showed more repetition and less novelty.

Overall, the project supports the use of a pairwise Potts model as a statistical generator for short amino-acid sequences. However, the conclusion should be limited to statistical sequencing. Future work should improve sampling diagnostics and test whether generated sequences are biologically meaningful.

References

[1] William P. Russ et al. "An Evolution-Based Model for Designing Chorismate Mutase Enzymes". In: *Science* (2020).

[2] Thomas A. Hopf et al. "Mutation effects predicted from sequence co-variation". In: *Nature Biotechnology* 35.2 (2017), pp. 128–135. DOI: 10.1038/nbt.3769.

[3] Simona Cocco et al. "Inverse statistical physics of protein sequences: a key issues review". In: *Reports on Progress in Physics* 81.3 (Jan. 2018), p. 032601. ISSN: 1361-6633. DOI: 10.1088/1361-6633/aa9965. URL: http://dx.doi.org/10.1088/1361-6633/aa9965.

[4] Marks Lab. *plmc: Inference of couplings in proteins and RNAs from sequence variation*. https://github.com/debbiemarkslab/plmc. GitHub repository. 2024.

[5] xqding. *DCA: PyTorch implementation of Direct Coupling Analysis*. https://github.com/xqding/DCA. GitHub repository. 2018.